

Agent 開發介紹

agents、hooks、skills……



Structure

架構

```
project/  
├── agents/  
├── skills/  
├── hooks/  
├── rules/  
├── references/  
└── memory/
```

agents/ —— 人設

skills/ —— 能力

hooks/ —— 週期事件＋閘門

memory/ —— 歷史紀錄＋狀態

agent: 人設 × 工具 × 必交物

Engineer

人設: 負責進行實驗、撰寫編輯程式

工具: Write · Edit · Bash

必交: 待辦 + 改了哪些檔 + 測試證據

Reviewer

人設: 不相信工程師, 跑測試寫報告

工具: Read + 跑測試

必交: 測試報告 + 判定

通用

人設: 一個 agent 專注做一種事情

工具: 能力邊界 = 工具清單

RULES

Agent 的行為規範

定義 Agent 應該如何行動，
做為執行任務時的約束。

例：.word 文件中的文字不可小於 11

REFERENCES

按需檢索的外部知識

提供詞彙表等內容，
需要時才讀取，不常駐於上下文中

例：詞彙表對照，例如：NCU=中央大學

Tool 和 Skill 的區別

Tool 定義「能做什麼」Skill 定義「如何做」

TOOL

可直接呼叫的功能

提供明確的輸入與輸出介面

負責執行單一操作

例: Read / Write / Bash / Grep

SKILL

封裝「如何完成任務」的可重用能力

定義何時使用哪些 Tools, 使用順序等

不一定是 Prompt, 也可以是程式化的 Workflow

將多個 Tools 組合成完整 Workflow

以 form-fill 為例

範例

```
---  
name: form-fill  
description: Complete web forms accurately and verify submission.  
---
```

Form Fill

1. Identify required fields
2. Fill using user-provided data
3. Check for validation errors
4. Submit
5. Verify success

frontmatter: 定義 Skill 的基本資訊

藉由在挑選 Skills 時, 只匯入 name 以及 description, 藉此來降低 Token 消耗。

若 Agent 選定 Skill 時, 才會完整匯入全部 內容至上下文中。

Hooks：在關鍵時刻自動介入 Agent 執行

SessionStart

開始前：準備

載入記憶、設定、System prompt

確保起始狀態正確

PreToolUse

操作前：檢查

檢查工具參數與安全風險

必要時阻止操作

Stop

結束前：驗證

檢查 TODO、測試與交付

未完成就要求繼續

Hook = 觸發條件 + 執行腳本

event: Stop

match: engineer

run: check-tests.js

timeout: 10 s

check-tests.js

1. 檢查是否有未完成 TODO
2. 檢查測試是否通過
3. 檢查是否還有未處理錯誤

→ 全部通過:ALLOW

→ 還沒完成:BLOCK

Trigger

Agent 準備結束任務時

Script

自動檢查 TODO、測試、錯誤

Decision

通過 -> 允許結束

失敗 -> 要求 Agent 繼續

LiteAgent: BEFORE_FINISH, 在任務結束前, 檢 查答案依據以及任務完成度

除了問「做完了沒」, 也問「**是否有根據**」

Evidence Check

答案可溯源

重要結論必須對應到
實際觀察到的頁面

Claim -> Evidence

Provenance Check

證據來源可信

引用的 URL / 頁面資訊
必須來自真實走訪軌跡。

Evidence -> Browser Trace

Completion Check

完成度審核

對照任務要求逐項檢查是否有缺漏

Requirement -> Verified

有根據 + 有來源 + 有完成 → 才能結束

每個任務都必須走完全部流程



不能跳過驗證

Agent 不能從「執行」直接進入「完成」

失敗就重新執行

驗證未通過 → 回到執行階段

重試有上限

避免 Agent 在同一路徑無限打轉

範例：登入任務

按下 “Log In” 不代表完成；必須確認帳號真的進入已登入狀態

模型負責決策, *Harness* 負責把任務做完

LOOP

讓任務持續推進

Observe→decide→act→check

沒完成就再跑一輪。

TOOLS

動作執行

定義能做什麼動作,

有什麼效果

CONTEXT

上下文

任務狀態、System prompt

每步操作結果、歷史軌跡等

GUARDRAILS

防止失控

Hooks、State Machine

限制錯誤操作, 避免假完成

模型可以替換, 但 Harness 會決定系統能否穩定

任務內狀態與跨任務經驗

Task Memory 任務進行中

- 已完成什麼
- 觀察到什麼
- 尚未完成的子任務

防止重做與迷路

Cross-task Memory 跨任務記憶

- 使用者偏好
- 成功的做法

防止每次重新學習網站如何操作

任務:

到 Booking 找到台北最便宜、評分 4.5 以上的飯店

Task State

Goal

找出台北最便宜、評分 ≥ 4.5 的飯店

Confirmed

- 地點: 台北
- 日期: 10/20–10/21
- 評分條件: ≥ 4.5
- 已排序: 價格由低到高

Observed

- Hotel A: NT\$2,180 / 4.6
- Hotel B: NT\$2,350 / 4.7

Progress

- 搜尋完成
- 篩選完成
- 尚未驗證 Hotel A 是否真的為最低價

Next

- 打開 Hotel A
- 驗證價格與評分
- 確認後才能回答

Context

Context 是預算, 不是倉庫

必要項目

System Prompt / Rules

Current Task State	目標、已完成、待完成、下一步
Recent Evidence	最近的 observation、tool result
Relevant Memory / Skills	目前任務有關的記憶與各 Skill 之描述

不必要項目

完整的歷史

過期的 Observation

重複資訊

舊有的內容不代表現在都需要

若狀態已經變動可以移除

同一個狀態內容等只留最新版

Memory 存放狀態; Context 用來處理現在要做什麼

Compression: 上下文壓縮

Compression: Context 不夠時, 有選擇地縮減低價值資訊

① Summarize: 舊對話 -> 摘要

保留:

決策結果、已確認事實

刪除:

重複內容、中間推理過程

過期 Observation

② Keep Structured

goal

confirmed

pending

evidence

todo

current_state

Dreaming: 把經歷整理成有用資訊

重新整理、抽取以及淘汰資訊, 提升資訊密度

Task running



Idle / Waiting



Dreaming



整理目前資訊

- 合併重複 observation
- 保留已確認事實
- 更新 task state
- 丟掉低價值細節



Cleaner Context

Task finished



Dreaming



抽取可重用資訊



Cross-Task Memory

CLIPROXY [連結](#)

Claude Code 中使用不同模型

Claude Code

→
Anthropic 協議

本地代理

→
OAuth / API key

Claude · Gemini · Chatgpt

為什麼需要換模型

不同模型, 各有擅長的任務

有些適合 Coding

有些適合 推理、審查

藉此達到更理想的結果

可自由串接以下模型

Claude / ChatGPT 訂閱登入 (OAuth: 免貼key)

Gemini · Grok · Kimi · GLM

OpenRouter · Ollama · VLLM

--models

```
$ claudemini --models
```

```
gemini-3.8-flash-high
```

```
gemini-3.7-flash-high
```

```
gemini-3.6-flash-high
```

```
gemini-3.5-flash-lite
```

```
gemini-3.1-flash-lite
```

```
...
```

```
$ claudeop --models
```

```
gpt-6-luna
```

```
gpt-5.6-luna
```

```
glm-5.3 · grok-4.7 · qwen3.8-max
```

```
deepseek-v4.1-flash · deepseek-v4-pro
```

```
...
```

Claude Code

子代理 vs 另開對話

SUBAGENT

不佔用上下文，處理雜事

同一 session 內派工，context 隔離

用完即丟，結果回傳就好

適合：搜尋、審查、探路等雜活

NEW SESSION

需要處理固定的不同任務

全新上下文

適合：長任務分段、不同角色、並行長跑

兩局靠交接條溝通，不靠記憶硬帶